

Java Programming Exercises

Object-Oriented Design Practice

Exercise 1 — Electronic Device Management

Build a Java console application that manages electronic devices sold in a store, using an in-memory ArrayList. The menu runs continuously until the user chooses to exit.

Abstract class `ElectronicDevice`

- Common fields: `deviceId`, `deviceName`, `manufacturer`, `sellingPrice`, and `importDate` (`LocalDate`, entered and displayed as `yyyy-MM-dd`).
- Declares the abstract method `displayInfo()`, overridden by every subclass. Each override must include the import date, not just the type-specific fields.

Static nested class `DeviceStatistics`

Nested inside `ElectronicDevice` and auto-updated whenever a device is added. Tracks the total device count, the Phone count, the Laptop count, and the average selling price across all devices.

Phone (extends `ElectronicDevice`)

- Adds `batteryCapacity` (must be greater than 0) and a boolean flag indicating whether the phone supports 5G.
- Overrides `displayInfo()`. Implements `Discountable` with a 10% discount.

Laptop (extends `ElectronicDevice`)

- Adds `ram` (must be greater than 0) and `screenSize`.
- Overrides `displayInfo()`. Implements `Discountable` with a 15% discount.

Discountable interface

Declares `double calculateDiscountPrice()`; Phone returns `sellingPrice * 0.90`; Laptop returns `sellingPrice * 0.85`.

Custom exceptions

- `DuplicateDeviceIdException` — the entered device ID already exists.
- `InvalidPriceException` — the entered selling price is not greater than 0.
- `InvalidAttributeException` — battery capacity (Phone) or RAM (Laptop) is not greater than 0.
- `DeviceNotFoundException` — declared for the case where no device matches a requested ID. None of the menu functions below need to throw it, since the one ID-based search (function 2) returns an empty `Optional` rather than throwing; it remains part of the exception design even though nothing currently triggers it.

- Also catch `InputMismatchException` wherever a numeric field is read from the keyboard, so entering non-numeric text does not crash the program — re-prompt instead.

Menu functions

- 1 — Add a device (Phone or Laptop): validate that the ID is not a duplicate (`DuplicateDeviceIdException`), that the price is greater than 0 (`InvalidPriceException`), and that the type-specific attribute is valid (`InvalidAttributeException`); collect the import date as `yyyy-MM-dd`.
- 2 — Search for a device by ID: returns `Optional<ElectronicDevice>` via the Stream API. Display the device if present, otherwise show a "not found" message — this path never throws.
- 3 — Display the list of phones that support 5G.
- 4 — Display the list of laptops with RAM of 16GB or more.
- 5 — Find the phone with the largest battery capacity; if there are no phones, show an appropriate message instead of an error.
- 6 — Find the laptop with the largest screen size; if there are no laptops, show an appropriate message instead of an error.
- 7 — Find the device with the highest selling price, using Stream + `Optional`; if the list is empty, show a message instead of an error.
- 8 — Compute the total value of all devices.
- 9 — Find devices by manufacturer (the user enters the manufacturer's name).
- 10 — Display the list of device names, built using `.map()`.
- 11 — Display the list of devices imported in a year specified by the user.
- 12 — Display the discounted price of every device: ID, name, original price, discounted price.
- 13 — Sort devices by name (A to Z). 14 — Sort devices by selling price, ascending. Both are implemented with a Lambda Expression passed directly to `Collections.sort()` / `List.sort()` — no standalone Comparator class.
- 15 — Display device statistics via `DeviceStatistics`: total number of devices, number of Phones, number of Laptops, and average selling price. With no devices in the list, show 0 rather than dividing by zero.

General requirements

- Use inheritance to reuse common attributes; Phone and Laptop implement `Discountable`.
- Apply Lambda Expressions and the Stream API to every filtering, searching, sorting, and computing operation above, replacing traditional for-loops. Use method references in place of lambdas wherever that is a clean fit, such as a bare getter reference in a comparator or in `.map()`.
- Every search returns `Optional`, never `null`.

Exercise 2 — Transportation Management System

Build a Java console application that manages a transportation center's vehicles. Car and Motorcycle objects are stored together in a single ArrayList, and the menu runs continuously until the user chooses to exit.

Abstract class `Vehicle`

- Common fields: `vehicleId`, `vehicleName`, `manufacturer`, `vehicleValue`, and `registrationDate` (`LocalDate`, entered as `yyyy-MM-dd`). On a parse failure, catch it and re-prompt with a message such as "Invalid date format. Please enter the date using `yyyy-MM-dd`."
- Implements `Comparable<Vehicle>`, with natural ordering by vehicle value, ascending. Declares public abstract `double calculateAnnualTax()`; overridden by each subclass.
- Every subclass overrides a display method (e.g. `displayInfo()`) showing its own type-specific fields; call it polymorphically through a `Vehicle` reference wherever possible, rather than casting to the concrete type first.

Static nested class `VehicleStatistics`

Auto-updated whenever a vehicle is added. Exposes public static methods `getTotalVehicles()`, `getTotalCars()`, `getTotalMotorcycles()`, and `getTotalVehicleValue()`.

Non-static inner class `VehicleInsurance`

- Fields: insurance provider and coverage amount. Every `Vehicle` must contain exactly one `VehicleInsurance`, entered by the user at creation time and shown as part of `displayInfo()`.
- Because it is a non-static inner class, a `VehicleInsurance` cannot be constructed on its own — it requires an enclosing `Vehicle` instance, using syntax such as `someVehicle.new VehicleInsurance(provider, amount)` from outside the class, or simply `new VehicleInsurance(provider, amount)` from inside one of `Vehicle`'s own constructors.

`car` (extends `Vehicle`)

- Adds `numberOfSeats` (must be greater than 0) and `fuelType` (enum: `Gasoline` / `Diesel` / `Electric`).
- `calculateAnnualTax()`: `Gasoline` → 5% of vehicle value, `Diesel` → 6%, `Electric` → 3%.
- `getRegistrationStatus()` (from `Registrable`): `Electric` → "Green Vehicle"; `Gasoline` or `Diesel` → "Standard Vehicle".

`Motorcycle` (extends `Vehicle`)

- Adds `engineCapacityCc` (must be greater than 0) and a boolean flag indicating whether the motorcycle has ABS.
- `calculateAnnualTax()`: under 150cc → 2% of vehicle value, 150cc or more → 4%.
- `getRegistrationStatus()`: ABS supported → "Safety Certified"; no ABS → "Basic Certified".

Registrable interface

Declares `String getRegistrationStatus();`, implemented by both `Car` and `Motorcycle`.

Custom exceptions

- `DuplicateVehicleIdException` — the entered vehicle ID already exists.
- `InvalidVehicleValueException` — vehicle value is 0 or less, number of seats is 0 or less, or engine capacity is 0 or less (one shared exception covering all three triggers).
- Handle every exception gracefully: show a clear message and let the user re-enter the value. The program must never crash on invalid-type keyboard input either.

Menu functions

1 — Add a Car. 2 — Add a Motorcycle. Both are validated against `DuplicateVehicleIdException` and `InvalidVehicleValueException`, and both collect a registration date and `VehicleInsurance` information.

3 — Display all vehicles.

4 — Search by ID. 5 — Search by name. Both use the Stream API and return `Optional<Vehicle>` (never null); show full information if found, otherwise "Vehicle not found."

6 — Display all electric cars. 7 — Display all motorcycles with ABS. 8 — Display vehicles by manufacturer.

9 — Sort by value, ascending, via natural ordering (`Comparable<Vehicle>`). 10 — Sort by vehicle name (A to Z). 11 — Sort by manufacturer (A to Z). 12 — Sort by annual tax, highest to lowest. All four sorts use a Lambda Expression passed directly to `Collections.sort()` / `List.sort()` — no standalone `Comparator` classes.

13 — Display a vehicle tax report: for every vehicle, show ID, name, type, annual tax, and registration status, via polymorphic calls to `calculateAnnualTax()` and `getRegistrationStatus()`.

14 — Display vehicle statistics via `VehicleStatistics`: total vehicles, total cars, total motorcycles, total vehicle value.

15 — Display vehicles registered within the last N days: the user enters N, and the program shows vehicles whose registration date falls within that many days of today.

General requirements

- Vehicle IDs must be unique. Use polymorphism by storing both `Car` and `Motorcycle` in a single `ArrayList<Vehicle>`.
- Use method references in place of lambdas wherever that is a clean fit.

Exercise 3 — Library Document Management

A library manages Books, Magazines, and Newspapers. Every document shares a unique Document ID, a Publisher Name, and a Number of Copies Issued. Books additionally track an Author Name and a Number of Pages; Magazines track an Issue Number and an Issue Month; Newspapers track an Issue Date.

Class design

- An interface, `IDisplayable`, declaring `void display()`; — the contract every document type fulfills.
- An abstract class `Document` implementing `IDisplayable`, holding the three shared fields and a constructor that sets them. `Book`, `Magazine`, and `Newspaper` extend `Document`, each with its own constructor (calling `super(...)` for the shared fields) and each overriding `display()` to print the shared fields plus its own additional fields.

DocumentManager class

- Add a new document (`Book`, `Magazine`, or `Newspaper`) — reject a Document ID that is already in use, with a clear message, and let the user re-enter it.
- Remove a document by Document ID — show a clear message when the ID does not match any stored document.
- Display information about a document — prints the full stored collection: every document currently held, each shown via its own `display()`, as distinct from the type-filtered search below.
- Search for documents by type (`Book`, `Magazine`, or `Newspaper`) and display the matches, or a message when none match.
- Exit the program.

Exercise 4 — University Admissions

University entrance exam candidates sit for one of three exam groups: A, B, or C. Every candidate has a Candidate Number, Full Name, Address, and Priority Level. Group A candidates take Math, Physics, and Chemistry; Group B candidates take Math, Chemistry, and Biology; Group C candidates take Literature, History, and Geography.

Class design

- An abstract class `Candidate` holding the four shared fields. The priority level is stored and displayed only — its exact Java type is not prescribed; choose one appropriate for that purpose.
- Three subclasses, one per exam group — `GroupACandidate`, `GroupBCandidate`, and `GroupCCandidate` — each adding three double score fields named for its own subjects: `GroupACandidate` has `mathScore`, `physicsScore`, `chemistryScore`; `GroupBCandidate` has `mathScore`, `chemistryScore`, `biologyScore`; `GroupCCandidate` has `literatureScore`, `historyScore`, `geographyScore`.
- `Candidate` declares an abstract method `calculateTotalScore()`, which each subclass overrides to sum its own three scores.

AdmissionManager class

- Add a new candidate — reject a Candidate Number that is already in use, with a message, and let the user re-enter it.
- Display a candidate's information along with their exam group, subject scores, and total score (via `calculateTotalScore()`).
- Search for a candidate by Candidate Number — show a clear message when no candidate matches.
- Exit the program.

Exercise 5 — Document Management

Build a small console application that manages a persistent collection of books.

IDocument interface

Declares `void input();` and `void display();`.

Book class

- Implements `IDocument`. Private fields: `id (int)`, `bookTitle (String)`, `authorName (String)`, `price (float)`.
- A no-argument constructor and a 4-argument constructor; getter and setter methods for all four fields.
- `input()`: prompts for and sets all four fields from the keyboard. `display()`: prints all four fields.

DocumentManager class

- Keeps the book collection in a field that persists and grows across calls — never replaced or reset — so that multiple "Add new book" calls accumulate rather than erase previously entered books.
- `addDocument()`: asks for the number of books `n` to add in this call, then collects `n` books' information, extending the persistent collection by those `n` books.
- `displayAllDocuments()`: prints every book currently stored, via each one's `display()`.
- `searchByAuthor(String authorName)`: prints every book by that author, or a "not found" message if there are none.

DocumentTest class and menu

1. Add new book
2. Display all books
3. Search books by author
4. Exit

Menu behaviour

- 1 — call `addDocument()`, then return to the menu.
- 2 — call `displayAllDocuments()`, then return to the menu.
- 3 — prompt for an author's name, call `searchByAuthor()` with it, then return to the menu.
- 4 — end the program.

Exercise 6 — Outdoor Trip Management System

Build an outdoor trip management system that applies object-oriented programming principles. The system manages trips such as family trips and adventure trips, tracking attributes such as trip ID, destination, departure date, and number of participants.

Abstract class `Trip`

- Protected fields: `id (String)`, `destination (String)`, `date (String)`, `numberOfParticipants (int)`.
- A default constructor and a parameterized constructor; getter and setter methods for all fields.
- Two abstract methods: `public abstract void enterInfo();` and `public abstract void showInfo();`

`FamilyTrip` (extends `Trip`)

- Adds `hasChildren (private boolean)` and `isExtendedFamily (private boolean)`. `true` means the group includes extended family or relatives beyond the immediate family; `false` means immediate family only.
- A no-argument constructor and a 6-argument constructor (the 4 inherited fields plus these 2), using `super (...)` to set the inherited fields. Getter and setter methods for the two new fields.
- `enterInfo()`: prompts for every field, inherited and new. `showInfo()`: prints every field.

`AdventureTrip` (extends `Trip`)

- Adds `activityType (private String)` and `difficultyLevel (private int)`.
- A no-argument constructor and a 6-argument constructor, using `super (...)` for the 4 inherited fields. Getter and setter methods for the two new fields.
- `enterInfo()` and `showInfo()` follow the same pattern as `FamilyTrip`.

`estimateDuration()`

- A method on `AdventureTrip` that calculates the estimated trip duration from the activity type and difficulty level: Hiking → 1 day per difficulty level; Rafting → 1.5 days per difficulty level; Mountain Climbing → 2 days per difficulty level. Match `activityType` against these three names case-insensitively.
- If `activityType` matches none of the three, return 0 and treat the activity type as unrecognized rather than computing a meaningless number.
- After `showInfo()` runs (as part of menu option 4 below), also print the estimated duration for that trip.

Example: for a Hiking trip with difficulty 4, the estimated duration is 4 days (1 day × 4).

```
Trip ID: TR001
Destination: Grand Canyon
Activity Type: Hiking
Difficulty Level: 4
Estimated Trip Duration: 4 days
```


TripTest class and menu

Please select:

1. Enter information for n family trips.
 2. Enter information for n adventure trips.
 3. Display information for n family trips (sorted by number of participants, descending).
 4. Display information for n adventure trips (sorted by difficulty level, ascending).
 5. Exit.
- Your choice:

Menu behaviour

- 1 — ask how many family trips to enter, then collect that many into a FamilyTrip array.
- 2 — ask how many adventure trips to enter, then collect that many into an AdventureTrip array.
- 3 — display the FamilyTrip array sorted by number of participants, descending.
- 4 — display the AdventureTrip array sorted by difficulty level, ascending, with each entry followed by its estimated duration.
- 5 — exit.

Exercise 7 — Hotel Management

Manage guests renting rooms at a hotel. Room type A costs \$500 per night, room type B costs \$300 per night, and room type C costs \$100 per night. For each guest, the hotel must track full name, age, and ID number, along with their room type and number of nights.

PersonAbs and Person

- Abstract class `PersonAbs`, with fields `name` and `age`, declaring `goToWork()` as abstract.
- Class `Person` extends `PersonAbs`, adding an `idNumber` attribute, and implementing `goToWork()` by printing exactly "go to work".

IHotel interface and Hotel class

- Interface `IHotel` with three methods: `void addCustomer(Person guest, String roomType, int nights);`, `void deleteCustomer(String idNumber);`, and `double calculatePrice(String idNumber);`
- Class `Hotel` implements `IHotel`. For each guest, it stores their `Person` information together with the room type and number of nights chosen for the current stay — these are per-stay details, not permanent facts about the person, so they are tracked alongside the guest record rather than inside `Person` itself.
- Reject a room type outside A, B, or C and re-prompt.

Add, remove, and calculate charges

- Add a new guest, together with their room type and number of nights — reject an ID number that is already in use, with a message, and let the user re-enter it.
- Remove a guest by ID number — show a clear message when the ID number does not match any stored guest.
- Calculate a guest's room charge, looked up by ID number: number of nights $\times$ the price per night for their room type (\$500 / \$300 / \$100 for A / B / C). Show a clear message when the ID number does not match any stored guest.

Exercise 8 — Transportation Management

The traffic police department needs to manage transportation vehicles: cars, motorcycles, and trucks.

IVehicle interface

Declares `void startEngine();`, `void stopEngine();`, and `void brake();`

Vehicle class (implements IVehicle)

- Common fields: `id`, `manufacturer`, `yearOfManufacture`, `sellingPrice`, `color`.
- Provides one shared implementation of `stopEngine()` (e.g. printing "Stopping engine") and `brake()` (e.g. printing "Braking"), inherited as-is by all three vehicle types. Only `startEngine()` needs type-specific behavior.

Car, Motorcycle, and Truck (extend Vehicle)

- Car adds `numberOfSeats (Integer)` and `engineType (String)`.
- Motorcycle adds `enginePower (Integer)`.
- Truck adds `loadCapacity (Integer)`.
- Each overrides `startEngine()` with its own message: Motorcycle prints "Starting motorcycle engine", Car prints "Starting car engine", Truck prints "Starting truck engine".

Truck.loadCargo(Integer weight)

If `weight` exceeds `loadCapacity`, print "Load weight is not suitable" and return the excess (`weight - loadCapacity`). Otherwise, print "Load weight is suitable" and return 0.

Example: for a truck with a 1000 kg load capacity, `loadCargo(1200)` prints "Load weight is not suitable" and returns 200; `loadCargo(800)` prints "Load weight is suitable" and returns 0.

Car.boardPassengers(Integer numberOfPassengers)

If `numberOfPassengers` exceeds `numberOfSeats`, print a message noting how many passengers do not fit; otherwise print a message noting that everyone fits. Return the number of passengers exceeding capacity, or 0 if they all fit.

Example: for a car with 4 seats, `boardPassengers(6)` reports 2 passengers over capacity and returns 2; `boardPassengers(3)` reports that everyone fits and returns 0.

VehicleManager class

- Add vehicles of any of the three types, held together in one `List<Vehicle>` — reject an ID that is already in use, with a message, and let the user re-enter it. Remove a vehicle by ID.
- Call `loadCargo` on a truck by ID — if the ID belongs to a Car or Motorcycle instead, report that it is not a truck rather than crashing with a cast exception.
- Call `boardPassengers` on a car by ID — the same not-the-right-type check as above.
- Search vehicles by manufacturer or by color.

- Show a clear message whenever an ID given to remove, loadCargo, or boardPassengers does not match any stored vehicle.
- Exit the program.

Exercise 9 — Warehouse Management

Build a warehouse management system that handles different product categories: food, electronics, and household goods.

IProduct interface and Product class

- Interface `IProduct` declares `void addStock(int quantity);`, `void removeStock(int quantity);`, and `void displayInfo();`
- Class `Product` implements `IProduct`, with fields `id (String)`, `name (String)`, `price (double)`, `quantity (int)`.
- `addStock()` increases quantity by the given amount. `removeStock()` decreases quantity by the given amount. Both reject a zero or negative requested amount as invalid. `removeStock()` additionally rejects an amount that exceeds the current quantity — print an error and leave quantity unchanged, rather than letting it go negative or silently clamping it.

Food, Electronics, and Household (extend Product)

- `Food` adds `expirationDate (String)`, `isPerishable (boolean)`, and `updateExpirationDate(String)`.
- `Electronics` adds `warrantyPeriod (String)`, `brand (String)`, and `extendWarranty(String additionalPeriod)`, which updates `warrantyPeriod` to reflect the extension — for example, replacing it with a combined description.
- `Household` adds `material (String)`, `usage (String)`, and `updateUsage(String)`.
- Each of the three overrides `displayInfo()` to show the shared `Product` fields plus its own — a `Food` display includes expiration and perishability information, and similarly for the other two types.

InventoryManager class

- Stores products in a `HashMap<String, Product>` keyed by ID, for efficient add, remove, and lookup.
- Add a product — reject an ID that is already in use. Remove a product by ID — show a clear message if the ID is not found.
- Find products by name, and separately, filter by category (food, electronics, or household).
- Call `updateExpirationDate` on a `Food` item by ID, `extendWarranty` on an `Electronics` item by ID, and `updateUsage` on a `Household` item by ID. In each case, if the ID belongs to the wrong category, report that rather than crashing on a bad cast; if the ID is not found at all, report that too.
- Display information for all products.

Main class

An interactive menu covering add, remove, search, and display, wired to the `InventoryManager` methods above.

Exercise 10 — Student Management

Build a small console application that manages a persistent collection of students.

IStudent interface

Declares `void input();` and `void display();`.

Student class

- Implements `IStudent`. Fields: `id (int)`, `fullname (String)`, `email (String)`, `mark (float)`.
- A no-argument constructor and a 4-argument constructor; getter and setter methods for all four fields.
- `input()`: prompts for and sets all four fields. `display()`: prints all four fields.

StudentManagement class

- `addStudent()`: collects exactly 3 `Student` objects per call. Keeps the underlying array as a persistent field that grows by 3 on each call, rather than being replaced, so that a second "Add New" call does not erase students entered earlier.
- `showStudent()`: iterates over and displays every stored student.
- `public Student sortStudentByMark()`: sorts the stored students by mark, descending, and returns the top-ranked (highest-mark) student after sorting.

MainClass and menu

```
=== MENU ===
1. Add New.
2. Show All.
3. Sort.
4. Exit.
Your choice:
```

Menu behaviour

- 1 — run `addStudent()`.
- 2 — run `showStudent()`.
- 3 — run `sortStudentByMark()` and print the student it returns.
- 4 — exit.

Exercise 11 — Book Library Management

Build a book library management system that handles different book categories: textbooks, novels, and reference books.

IBook interface and Book class

- Interface `IBook` declares `void addCopies(int quantity);`, `void removeCopies(int quantity);`, and `void displayInfo();`
- Class `Book` implements `IBook`, with fields `id (String)`, `title (String)`, `author (String)`, `price (double)`, `quantity (int)`.
- `addCopies()` increases quantity by the given amount. `removeCopies()` decreases quantity by the given amount, but rejects — with an error message, leaving quantity unchanged — a removal that would take quantity below 0.

Textbook, Novel, and ReferenceBook (extend Book)

- `Textbook` adds `subject (String)`, `academicLevel (String)`, and `updateAcademicLevel(String)`.
- `Novel` adds `genre (String)`, `isBestseller (boolean)`, and `markAsBestseller()` — no parameters; it simply sets `isBestseller` to `true`.
- `ReferenceBook` adds `fieldOfStudy (String)`, `edition (String)`, and `updateEdition(String)`.
- Each overrides `displayInfo()` to include the shared `Book` fields plus its own.

LibraryManager class

- Stores books in a `HashMap<String, Book>` keyed by ID.
- Add a book — reject an ID that is already in use. Remove a book by ID — show a clear message if the ID is not found.
- Find books by title, and separately, by category (textbook, novel, or reference).
- Call `updateAcademicLevel` on a `Textbook` by ID, `markAsBestseller` on a `Novel` by ID, and `updateEdition` on a `ReferenceBook` by ID. In each case, report a wrong-category ID rather than crashing on a bad cast, and report an ID that is not found at all.
- Display information for all books.

Exercise 12 — Hospital Doctor Management

Build a hospital staff management system covering two kinds of doctors: specialists and surgeons.

Abstract class Doctor

- Protected fields: `doctorId (String)`, `doctorName (String)`, `specialization (String)`, `salary (float)`.
- A default constructor and a parameterized constructor; getter and setter methods for all fields.
- Abstract methods: `public abstract void input();` and `public abstract void display();`

SpecialistDoctor and Surgeon (extend Doctor)

- `SpecialistDoctor` adds `expertise (private String)` and `yearsOfExperience (private int)`.
- `Surgeon` adds `numberOfSurgeries (private int)` and `hospitalName (private String)`.
- Each gets a no-argument constructor and a 6-argument constructor (the 4 inherited fields plus its own 2). The 6-argument constructor's first statement is `super(doctorId, doctorName, specialization, salary)`, followed by setting the two new fields directly.
- Getter and setter methods for the new fields. Both classes implement `input()` (prompts for every field, inherited and new) and `display()` (prints every field).

DoctorTest class and menu

```
Please select:
1. Enter information for n specialist doctors.
2. Enter information for n surgeons.
3. Display information for n specialist doctors (sorted by salary,
   descending).
4. Display information for n surgeons (sorted by number of
   surgeries, ascending).
5. Exit.
Your choice:
```

Menu behaviour

- 1 — collect n specialist doctors into a `SpecialistDoctor` array.
- 2 — collect n surgeons into a `Surgeon` array.
- 3 — display specialist doctors sorted by salary, descending.
- 4 — display surgeons sorted by number of surgeries, ascending.
- 5 — exit.

Exercise 13 — Computer & Book Product Catalog

Build a small product catalog covering two kinds of products: computers and books.

Abstract class Product

- Protected fields: `proId (String)`, `proName (String)`, `year (int)`, `price (float)`.
- Two constructors, getter and setter methods for the fields, and two abstract methods: `public abstract void input();` and `public abstract void display();`

Computer and Book (extend Product)

- Computer adds `speed (private String)` and `producer (private String)`.
- Book adds `type (private String)` and `publisher (private String)`.
- Each gets a no-argument constructor and a 6-argument constructor. The 6-argument constructor's first statement is `super(proId, proName, year, price)`, followed by setting the two new fields directly.
- Getter and setter methods for the new fields. Both classes implement `input()` and `display()`, covering inherited and new fields.

Test class and menu

```
Please select:
1. Input information for n Computers.
2. Input information for n Books.
3. Display information of n Computers by sorting the price
   descending.
4. Display information of n Books by sorting the publisher
   ascending.
5. Exit.
Your choice:
```

Menu behaviour

- 1 — collect `n` computers into a `Computer` array.
- 2 — collect `n` books into a `Book` array.
- 3 — display the `Computer` array sorted by price, descending.
- 4 — display the `Book` array sorted by publisher, ascending.
- 5 — exit.

Exercise 14 — Exception Handling Practice

Write eight small, independent Java programs, each one triggering and handling the named exception with try-catch-finally. None require a larger application around them.

1. Wrong input type

Prompt for a number with `Scanner.nextInt()` (or similar). Entering non-numeric text throws `InputMismatchException`. Catch it and print a clear message instead of a stack trace.

2. Divide by zero

Use `int` division for this program specifically: `5 / 0` throws `ArithmeticException` ("`/ by zero`"). `double` division by zero does not throw — `5.0 / 0.0` silently produces `Infinity` — so using `doubles` here leaves nothing to catch.

3. Opening a file that does not exist

Open a nonexistent path with `FileReader` or `FileInputStream`; this throws `FileNotFoundException`. Catch it and print a message.

4. Invalid array index

Access an array outside its bounds, for example `arr[arr.length]`; this throws `ArrayIndexOutOfBoundsException`. Catch it and print a message.

5. Non-numeric string to number

`Integer.parseInt("abc")` throws `NumberFormatException`. Catch it and print a message.

6. A method cannot find a file

Use `java.nio.file.Files.newBufferedReader(path)` to open a file path that does not exist; this throws `NoSuchFileException`. Catch it and print a message.

7. try-catch-finally with a throws declaration

Write a method declared `throws ArithmeticException` (or a broader `Exception`) whose body performs `int` division that can divide by zero. Call it from a `try` block, catch the exception, and use a `finally` block to print something that always runs, regardless of whether the division succeeded.

8. assert for a positive number

Use `assert value > 0 : "value must be positive";` on a value read from the keyboard. Java disables assertions by default — an `assert` statement silently does nothing at runtime unless the program is run with the `-ea` (enable assertions) JVM flag. Without that flag, this program will appear to do nothing even when the assertion should fail, which is expected behavior, not a bug.